In the query, we check for headaches and antibiotics, but the patient shouldn't be diabetic—the query returns John Doe as he isn't diabetic but has headaches and is on antibiotics.

The `flattened` data types come in handy, especially when we expect many fields on an ad hoc basis and defining mapping definitions for all of them beforehand isn't feasible. Keep in mind that the subfiles of a flattened field are always `keyword` types.

4.6.5 The join data type

If you are from a relational database world, you know the relationships between data—the joins that enable parent-child relationships. Every document indexed by Elasticsearch is independent and maintains no relationship with any other documents in that index. Elasticsearch denormalizes the data to achieve speed and gain performance during indexing and search operations. While we are advised to be cautious about maintaining and managing relationships in Elasticsearch, the `join` data type is available to create parent-child relationships if we need them.

Let's learn about `join` data by considering an example of doctor-patients relationships (one-to-many): one doctor can have multiple patients, and each patient is assigned to one doctor. To work with parent-child relationships using the `join` data type, we need to create a field that is the `join` type and add information via a `relations` object that mentions the relationship (in this case, the doctor-patient relationship). The following listing prepares the `doctors` index with a schema definition.

Listing 4.36 Mapping of the `doctors` schema definition

```
PUT doctors
{
  "mappings": {
    "properties": {
      "relationship":{
        "type": "join",
        "relations":{
          "doctor":"patient"
        }
      }
    }
  }
}
```

Declares a property as the join type

Names of the relations

The query has two important points to note:

- We declare a `relationship` property of type `join`.
- We declare a `relations` attribute and give the names of the relations (in this case, only one `doctor:patient` relation).

Once the schema is ready and indexed, let's index two types of documents: one representing the doctor (parent) and the other representing two patients (children). Here's the doctor's document, with the relationship named `doctor`.